

Miskolci Egyetem
Gépészmérnöki és Informatikai Kar
Informatikai Intézet

2025/2026. tanév II. félév

Trófea nyilvántartó rendszer
Web technológiák 2. (GEIAL334-B2)
Jegyzőkönyv

Készítette: Erdélyi Péter (JH3V7T)

Gyakorlatvezető: Dr. Agárdi Anita

Miskolc, 2026. június 22.

TARTALOMJEGYZÉK

1. FELADATLEÍRÁS.....	1
2. BEJELENTKEZÉS	2
3. ADATOK KILISTÁZÁSA	4
4. ADATOK LÉTREHOZÁSA	6
5. ADATOK MÓDOSÍTÁSA.....	8

1. FELADATLEÍRÁS

Jelen jegyzőkönyv tartalmazza a Web technológiák 2. című tantárgyból készített évközi feladatomat. Ebben a feladatban egy nyilvántartó rendszer fejlesztésével foglalkoztam, ahol az implementációhoz a következő technológiákat alkalmaztam:

- Angular
- Node.js
- MongoDB

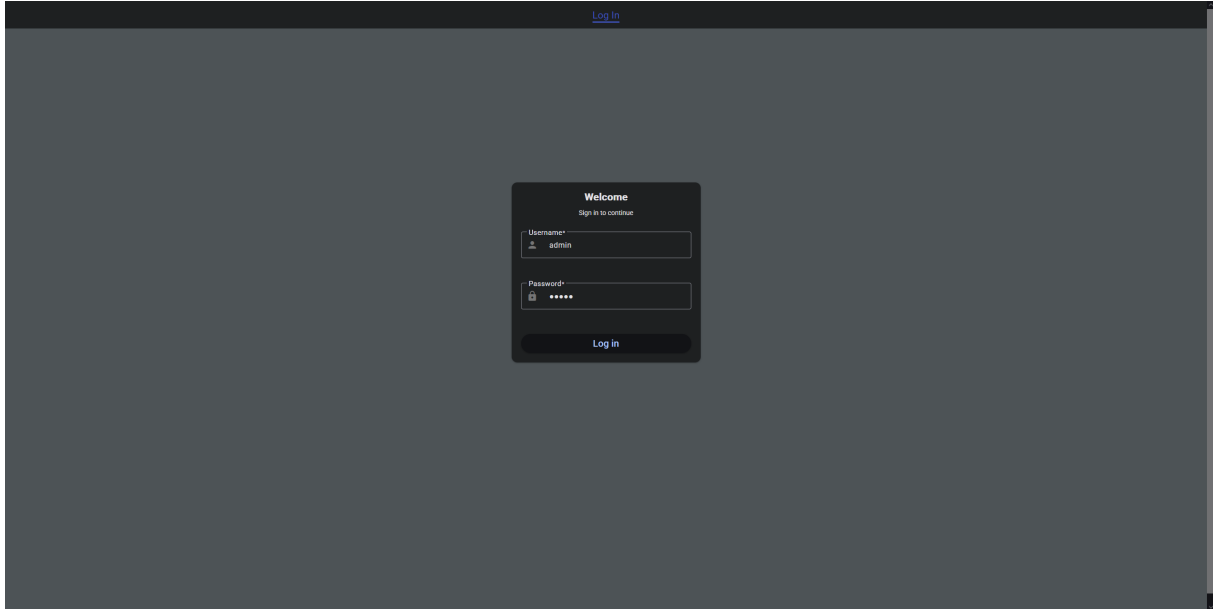
Az Achievement tracker adatbázis célja a különböző játékokban szerezhető trófeák kezelése, illetve ezeknek a felhasználókhoz való hozzárendelése. A projekt szerkezetileg lényegében két nagy részre bontható:

- `backend`
 - Bejelentkezés hitelesítése
 - HTTP kérések kiszolgálása
 - Kezdeti adatok feltöltése
- `frontend`
 - Műveletek: létrehozás, kilistázás, módosítás, törlés
 - Felhasználói felület megjelenítése

A jegyzőkönyv hátralévő részében röviden bemutatom a nyilvántartó rendszer működését és felhasználói felületét.

2. BEJELENTKEZÉS

A Trófea nyilvántartó rendszer használatához bejelentkezés szükséges. Ezt a felületet szemlélteti az 1. ábra.



1. ábra: Bejelentkezési felület

A belépéshez meg kell adni a felhasználónevet és jelszót, amiknek hitelességét a program ellenőrzi, hiányos vagy helytelen adatok esetén pedig figyelmeztetést kapunk. Fontos kiemelni, hogy a fő menüpontok elérhetősége is függ a bejelentkezési állapottól, vagyis az adatmanipuláció csak belépett felhasználóknak lehetséges. Az útvonalak védelme látható a 2. ábrán, ahol a különböző oldalak megnyitásának jogosultságát egy authGuard vizsgálja.

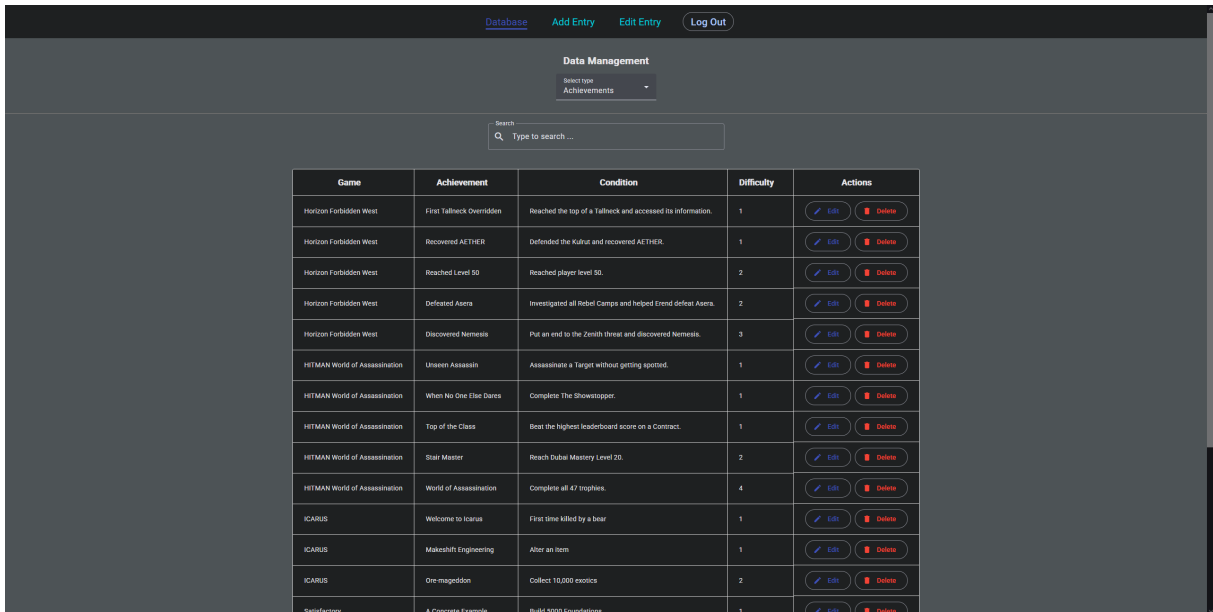
```
export const routes: Routes = [
  { path: '', redirectTo: 'login', pathMatch: 'full' },
  { path: 'login', component: LoginComponent },
  {
    path: 'data',
    canActivate: [authGuard],
    children: [
      { path: '', component: DataPageComponent },
      { path: ':type', component: DataPageComponent }
    ]
  },
  {
    path: 'add',
    canActivate: [authGuard],
    children: [
      { path: '', component: AddPageComponent },
      { path: ':type', component: AddPageComponent }
    ]
  },
  {
    path: 'edit',
    canActivate: [authGuard],
    children: [
      { path: '', component: EditPageComponent },
      { path: ':type', component: EditPageComponent },
      { path: ':type/:id', component: EditPageComponent }
    ]
  }
];
```

2. ábra: Útvonalak védelme

A felhasználói felület egységes és letisztult kinézetéhez a program számos részében az Angular Material könyvtárat vettem igénybe. Ennek segítségével jelenítem meg a mezőket (`mat-form-field`), az azokban lévő ikonokat (`mat-icon`), vagy éppen a hibaüzeneteket (`mat-error`).

3. ADATOK KILISTÁZÁSA

Sikeres belépést követően a felhasználó átirányításra kerül az adatbázis oldalra. Ennek fő feladata az adatok kilistázása, ugyanakkor lehetőség van módosítási vagy törlési műveletek kezdeményezésére is. Az adatbázis oldalát mutatja a 3. ábra.



The screenshot shows a web application interface for 'Data Management'. At the top, there are links for 'Database', 'Add Entry', 'Edit Entry', and 'Log Out'. Below these is a dropdown menu labeled 'Select type' with 'Achievements' selected. A search bar with the placeholder 'Type to search ...' is present. The main content is a table with the following data:

Game	Achievement	Condition	Difficulty	Actions
Horizon Forbidden West	First Tallneck Overridden	Reached the top of a Tallneck and accessed its information.	1	Edit Delete
Horizon Forbidden West	Recovered AETHER	Defended the Kalus and recovered AETHER.	1	Edit Delete
Horizon Forbidden West	Reached Level 50	Reached player level 50.	2	Edit Delete
Horizon Forbidden West	Defeated Asura	Investigated all Rebel Camps and helped Drend defeat Asura.	2	Edit Delete
Horizon Forbidden West	Discovered Nemesis	Put an end to the Zenith threat and discovered Nemesis.	3	Edit Delete
HITMAN World of Assassination	Unseen Assassin	Assassinate a Target without getting spotted.	1	Edit Delete
HITMAN World of Assassination	When No One Else Dares	Complete The Showstopper.	1	Edit Delete
HITMAN World of Assassination	Top of the Class	Beat the highest leaderboard score on a Contract.	1	Edit Delete
HITMAN World of Assassination	Star Master	Reach Dubai Mastery Level 20.	2	Edit Delete
HITMAN World of Assassination	World of Assassination	Complete all 47 trophies.	4	Edit Delete
ICARUS	Welcome to Icarus	First time killed by a bear	1	Edit Delete
ICARUS	Makeshift Engineering	Alter an item	1	Edit Delete
ICARUS	One mageddon	Collect 10,000 exotics	2	Edit Delete
Satisfactory	A Concrete Example	Build 5000 Foundations	1	Edit Delete

3. ábra: Adatok listázása

Legfelül található egy legördülő menü, ahol a megtekinteni kívánt adattípust lehet megjelölni. A három választható opció:

- tróféák
- felhasználók
- felhasználói tróféák (összeköti az előzőket)

Amikor kiválasztunk egy másik opciót, akkor az oldal dinamikusan újratölti a táblázatot a megfelelő tartalommal. Tulajdonképpen a `DataPage` komponensben egy switch-case szerkezet dönti el, hogy melyik másik komponens beágyazására van szükség (lásd 4. ábra). A listázáshoz kapcsolódó komponensek pedig az absztrakt `ListDataDirective` direktívából öröklődnek, ezáltal implementálják a szükséges metódusokat (pl. betöltés, szerkesztés). Érdemes megemlíteni, hogy ugyanezt az elvet követik a hozzáadásért és módosításért felelős komponensek is.

```
<div class="header-container">
  <h2>Data Management</h2>

  <mat-form-field appearance="fill">
    <mat-label>Select type</mat-label>
    <mat-select [value]="selectedView" (selectionChange)="onTypeChange($event.value)">
      <mat-option value="achievements">Achievements</mat-option>
      <mat-option value="userAchievements">User Achievements</mat-option>
      <mat-option value="users">Users</mat-option>
    </mat-select>
  </mat-form-field>
</div>

<hr />

@switch (selectedView) {
  @case ('achievements') {
    <app-list-achievements></app-list-achievements>
  }
  @case ('userAchievements') {
    <app-list-user-achievements></app-list-user-achievements>
  }
  @case ('users') {
    <app-list-users></app-list-users>
  }
}
```

4. ábra: A *DataPageComponent* felépítése

A „Database” lap néhány egyéb kényelmi funkciót is rejt magában. Egy keresőmező segítségével biztosítja az adatok közötti szűrést, valamint lehetővé teszi a táblázatban az oszlop szerinti rendezést. A rendezéshez a `MatSort` modult importáltam be, és alkalmaztam az `::ng-deep` CSS-szelektort a fejlécekre, hogy az oszlopcímeket középre tudjam igazítani (ezáltal felülírva a Material belső szabályrendszerét).

4. ADATOK LÉTREHOZÁSA

Az adatok hozzáadása az „Add Entry” menüpontra kattintva érhető el. Egy trófea létrehozását illusztrálja az 5. ábra.

The screenshot shows a web application interface for adding a new achievement. At the top, there are navigation links: 'Database', 'Add Entry', 'Edit Entry', and 'Log Out'. Below these is a section titled 'Add New Entry' with a dropdown menu set to 'Achievement'. The main form, titled 'Add Achievement', has the following fields: 'Game' (with 'Diablo' entered), 'Achievement name' (empty), 'Required' (empty), 'Condition' (with 'Eliminate Diablo, the Lord of Terror' entered), and 'Difficulty' (with '4' selected from a dropdown). An 'Add' button is located at the bottom of the form.

5. ábra: Trófea létrehozása

Első lépésben ki kell választani az adattípust a korábban már bemutatott legördülő menüből, ami után a `selectedView` gondoskodik róla, hogy az idetartozó űrlap jelenjen meg előttünk. Ezt követően ki kell tölteni a szükséges adatmezőket, majd az „Add” gombra nyomva létrejön az egyed az adatbázisban. Ha valamelyik kötelező mező üresen marad, akkor ezt egy hibaüzenet jelzi, ami egyúttal letiltja az elküldést is.

Az űrlap elkészítését a `DataFormComponent` végzi, amely inputként fogadja a cím- és gombfeliratokat, illetve az adatmezők csoportját. Kimenete egy `formSubmit` objektum, amely gombnyomáskor egy eseményt bocsát ki. Az ehhez kapcsolódó `executeSubmit()` metódust az `add` és `edit` komponensek implementálják. Mivel ezek az oldalak ugyanazt az űrlapelemet hasznosítják újra, így a megjelenés konzisztens marad. Az űrlap működését demonstrálják az alábbi 6 – 8. ábrák.

```
<mat-card>
  <h2>{{ title }}</h2>
  <form [formGroup]="formGroup" (ngSubmit)="formSubmit.emit()">
    <ng-content></ng-content>
    <button mat-raised-button class="submit-button" type="submit" color="primary" [disabled]="formGroup.invalid">
      {{ submitButtonText }}
    </button>
  </form>
</mat-card>
```

6. ábra: Űrlap felépítése

```
mat-card {
  display: flex;
  flex-direction: column;
  width: 100%;
  max-width: 600px;
  margin: 2rem auto;
  padding: 20px;
  background-color: #1e2021;
}

h2 {
  margin: 0 0 1.5rem 0;
  text-align: center;
}

form {
  display: flex;
  flex-direction: column;
  width: 100%;
}

.submit-button {
  display: flex;
  margin: 0 auto;
  font-size: 1.25rem;
}
```

7. ábra: Űrlap formázása

```
import { Directive } from '@angular/core';
import { FormGroup } from '@angular/forms';

@Directive()

export abstract class ManageDataDirective {
  abstract title: string;
  abstract submitButtonText: string;
  abstract dataForm: FormGroup;

  onSubmit(): void {
    if (this.dataForm.invalid) {
      return;
    }

    this.executeSubmit();
  }

  abstract executeSubmit(): void;
}
```

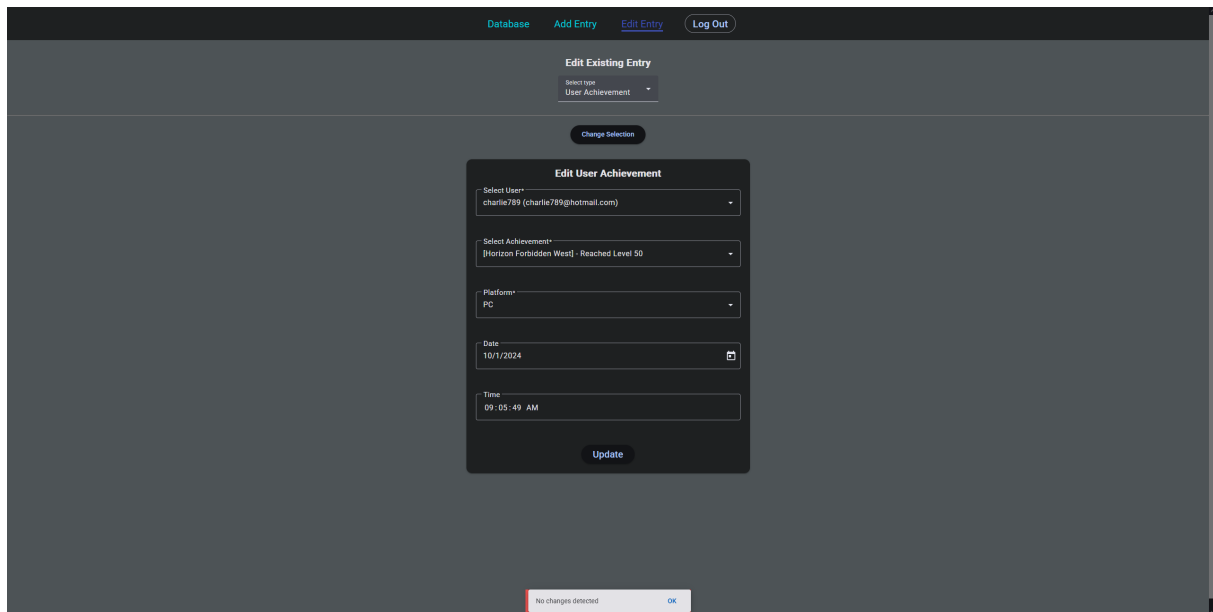
8. ábra: Adatkezelési direktíva

5. ADATOK MÓDOSÍTÁSA

A szerkesztési művelet kétféleképpen is indítható:

- az adatbázis oldalon valamely egyed sorában az „Edit” gombra kattintva
- az „Edit Entry” menüpontból

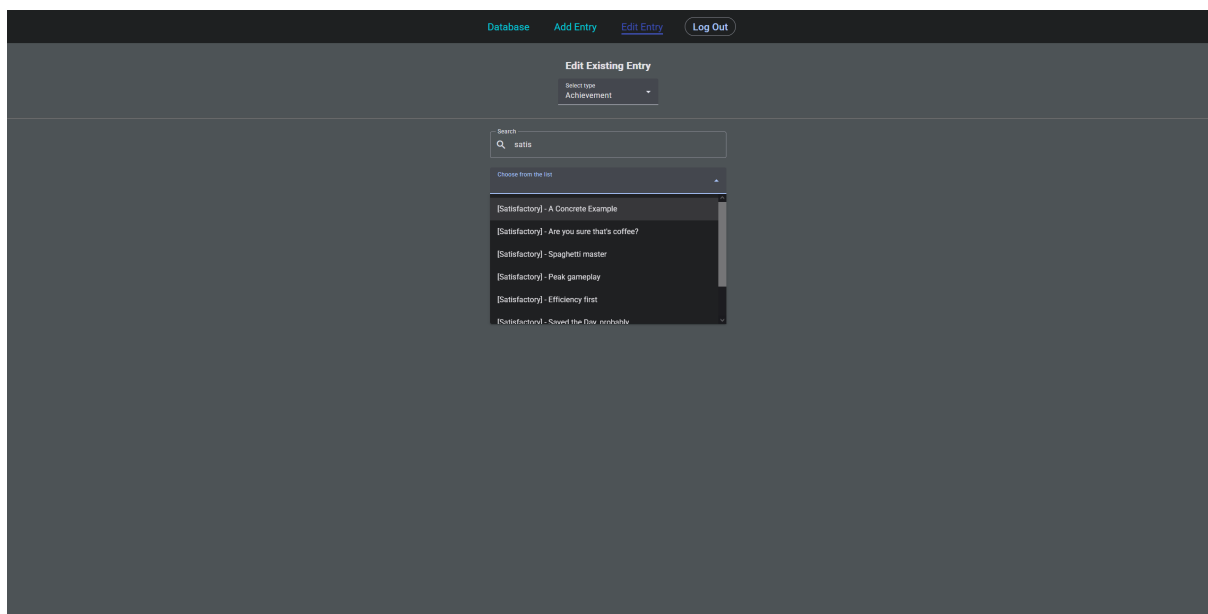
Tekintsük az első eshetőséget, amikor az adatmódosítást a táblázatból aktiváljuk. Erre mutat példát a 9. ábra.



9. ábra: Adatmódosítás az „Edit” gombról érkezve

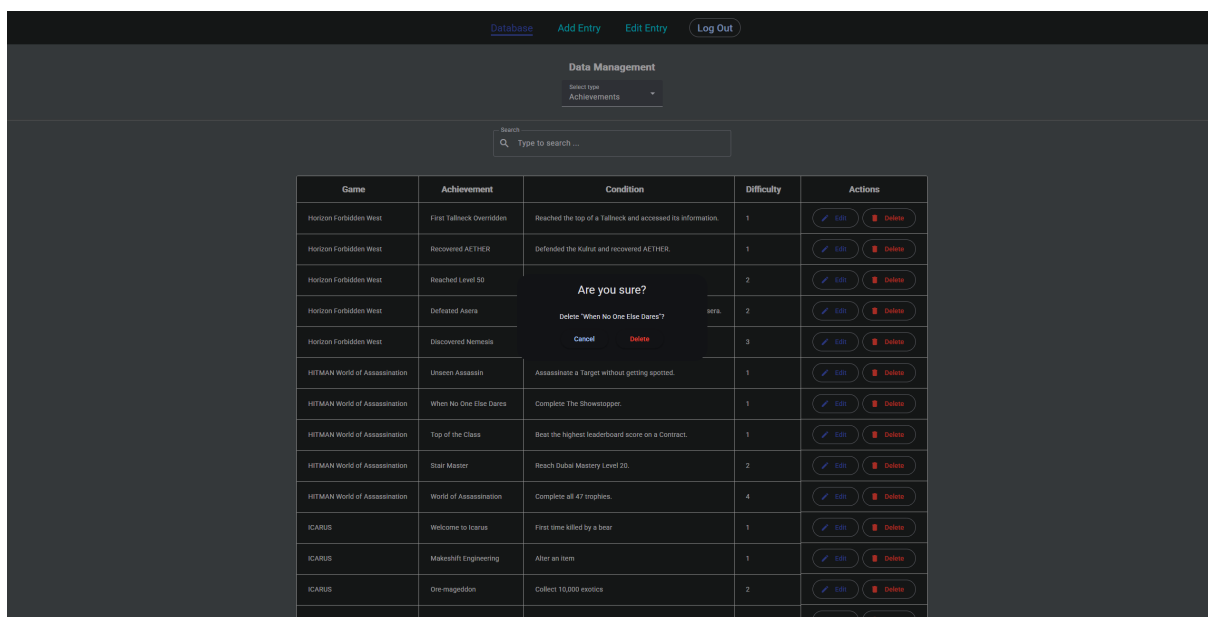
Ebben az esetben alapvetően az adatlétrehozás menüjéhez hasonló űrlap fogad minket, azzal a lényegi különbséggel, hogy a szerkeszteni kívánt adatokat a program már előre kitölti nekünk, így elegendő csak a módosítandó mező(ke)t átírni. Az „Update” gomb végzi az adatok frissítését. Ha véletlenül változatlanul marad minden, akkor a frissítés nem hajtodik végre, hanem egy értesítést kapunk a hibáról. Ezeket az értesítéseket a MatSnackBarBar modul segítségével generálom, amihez pedig a notificationService szolgáltatást injektálom.

A szerkesztés másik módja közvetlen az „Edit Entry” menüpont használata. Mivel a változtatáshoz a programnak ismernie kell az egyed azonosítóját, ezért itt először szűkíteni kell az adathalmazt. A felső legördülő listából kiválasztjuk a típust, majd egy második menüből a pontos egyedet. A keresőmezővel lehet a listaelemeket szűrni (lásd 10. ábra). Amint a kiválasztás megtörténik, úgy a módosítás innentől voltaképpen megegyezik az előbb felvázolt folyamattal. A „Change Selection” lehetőséget ad egy másik egyed adatainak szerkesztésére anélkül, hogy az oldalt újra kellene tölteni.



10. ábra: Adatmódosítás közvetlenül az „Edit Entry” menüpontból

Törlés az adatbázis oldalon a „Delete” gombbal valósítható meg. Ekkor egy párbeszédablak jelenik meg és megerősítést kér erről, mert ez a művelet nem vonható vissza. Ezt mutatja a 11. ábra.



11. ábra: Törlési művelet